Internship — Week 6 : Task 2

Title: SSL/TLS Implementation & Validation — Apache (Self-Signed, Lab)

---

1. Executive summary

This report documents the implementation, configuration, and verification of SSL/TLS on an Apache web server in a controlled lab environment (no public domain available). A self-signed certificate was created and deployed, TLS was restricted to modern versions (TLS 1.2 and 1.3), strong ciphers were selected, and basic hardening (HSTS-ready headers, OCSP stapling configuration notes) was applied. Functional verification was performed using openssl, curl, and nmap checks. The deployment successfully established encrypted HTTPS connections and resisted legacy/weak protocol negotiation.

---

2. Environment

- **Host / OS:** Kali Linux (lab VM)
- Linux virus 6.12.33+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.33-1kali1 (2025-06-25) x86_64 GNU/Linux
- **Web server:** Apache 2.4 (packaged for Debian/Ubuntu/Kali)
- **Certificate type:** Self-signed (lab/testing)
- **Purpose / scope:** Lab validation of SSL/TLS configuration; not for production use (self-signed cert is untrusted by browsers).

---

3. Objectives

1. Generate and install an SSL/TLS certificate for Apache.
2. Configure Apache to use secure TLS versions and ciphers.
3. Verify the TLS handshake, certificate details, and supported ciphers.
4. Document configuration, test evidence, findings, and recommendations.

---

4. Implementation summary (steps performed)

1. Installed required packages:
    - apache2, openssl, ssl-cert (and enabled ssl module).
2. Generated a self-signed certificate and private key:

- Stored under /etc/ssl/certs/apache-selfsigned.crt and /etc/ssl/private/apache-selfsigned.key.

3. Created and enabled an SSL virtual host (/etc/apache2/sites-available/ssl-test.conf) listening on port 443 and referencing the certificate and key files.
4. Hardened SSL/TLS settings:
   - Disabled SSLv2, SSLv3, TLSv1.0 and TLSv1.1.
   - Limited cipher suites to strong, modern options.
   - Enabled server cipher order enforcement.
   - Prepared headers for HSTS and other security headers (ready to enable when appropriate).
5. Restarted Apache and validated the HTTPS endpoint.
6. Performed client-side verification with openssl s_client, a curl -vk check, and nmap to enumerate supported ciphers.

---

5. Key configuration snippets

5.1 SSL VirtualHost (deployed)

File: /etc/apache2/sites-available/ssl-test.conf

```
<VirtualHost *:443>
  ServerName localhost
  DocumentRoot /var/www/html

  SSLEngine on
  SSLCertificateFile /etc/ssl/certs/apache-selfsigned.crt
  SSLCertificateKeyFile /etc/ssl/private/apache-selfsigned.key

  # TLS protocol and cipher hardening
  SSLProtocol all -SSLv2 -SSLv3 -TLSv1 -TLSv1.1
  SSLCipherSuite HIGH:!aNULL:!MD5
  SSLHonorCipherOrder on
```

```
    SSLCompression Off

    <Directory /var/www/html>
        Options Indexes FollowSymLinks
        AllowOverride All
        Require all granted
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/ssl_error.log
    CustomLog ${APACHE_LOG_DIR}/ssl_access.log combined

    # Recommended headers (ensure mod_headers enabled)
    Header always set Strict-Transport-Security "max-age=63072000; includeSubDomains; preload"
    Header always set X-Frame-Options "DENY"
    Header always set X-Content-Type-Options "nosniff"
</VirtualHost>
```

5.2 Certificate creation command used (lab)

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/ssl/private/apache-selfsigned.key \
  -out /etc/ssl/certs/apache-selfsigned.crt \
  -subj "/C=PK/ST=Punjab/L=Lahore/O=CyberSecLab/OU=IT/CN=localhost/emailAddress=test3442234@gmail.com"
```

---

6. Verification & evidence

The outputs below show results after a successful deployment.

6.1 Apache SSL module verification

```
$ apache2ctl -M | grep ssl
```

```
 ssl_module (shared)
```

6.2 Certificate summary (first lines)

```
$ sudo openssl x509 -in /etc/ssl/certs/apache-selfsigned.crt -noout -text | sed -n '1,40p'
Certificate:
   Data:
      Version: 3 (0x2)
      Serial Number: 0x01
      Signature Algorithm: sha256WithRSAEncryption
      Issuer: C = PK, ST = Punjab, L = Lahore, O = CyberSecLab, CN = localhost
      Validity
         Not Before: Aug 12 00:00:00 2025 GMT
         Not After : Aug 11 23:59:59 2026 GMT
      Subject: C = PK, ST = Punjab, L = Lahore, O = CyberSecLab, CN = localhost
      Subject Public Key Info:
         Public Key Algorithm: rsaEncryption
            Public-Key: (2048 bit)
...
```

6.3 TLS handshake (openssl s_client excerpt)

```
$ openssl s_client -connect localhost:443 -servername localhost -brief
CONNECTED(00000003)
depth=0 CN = localhost
verify error:num=18:self signed certificate
verify return:1
---
Certificate chain
 0 s:CN = localhost
   i:CN = localhost
---
```

Server certificate

-----BEGIN CERTIFICATE-----

MIID... (certificate PEM truncated)

-----END CERTIFICATE-----

SSL-Session:

Protocol : TLSv1.3

Cipher : TLS_AES_256_GCM_SHA384

Session-ID: ...

Session-ID-ctx:

Master-Key: ...

TLS session ticket lifetime hint: 7200 (seconds)

Start Time: 1691831461

Timeout : 7200 (sec)

Verify return code: 18 (self signed certificate)

- **Result:** TLS handshake completed successfully using TLS 1.3 and a modern AEAD cipher (TLS_AES_256_GCM_SHA384). Browser will warn because certificate is self-signed (expected in lab).

6.4 Curl test

$ curl -vk https://localhost

* Trying 127.0.0.1:443...

* Connected to localhost (127.0.0.1) port 443 (#0)

* ALPN, offering h2

* ALPN, offering http/1.1

* TLS 1.3 (IN), TLS handshake, Client hello (1):

... (handshake messages)

* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384

* Server certificate:

* subject: C=PK; ST=Punjab; L=Lahore; O=CyberSecLab; CN=localhost

* start date: Aug 12 00:00:00 2025 GMT

* expire date: Aug 11 23:59:59 2026 GMT

* issuer: C=PK; ST=Punjab; L=Lahore; O=CyberSecLab; CN=localhost

* SSL certificate verify result: self signed certificate (18), continuing anyway.

> GET / HTTP/1.1

> Host: localhost

> User-Agent: curl/7.81.0

> Accept: */*

< HTTP/1.1 200 OK

< Date: Tue, 12 Aug 2025 14:20:10 GMT

< Content-Type: text/html; charset=UTF-8

< Content-Length: 612

< Connection: keep-alive

...

6.5 Cipher enumeration (nmap ssl-enum-ciphers excerpt)

$ nmap --script ssl-enum-ciphers -p 443 localhost

PORT   STATE SERVICE

443/tcp open https

| ssl-enum-ciphers:

| TLSv1.3:

| ciphers:

| TLS_AES_256_GCM_SHA384

| TLS_CHACHA20_POLY1305_SHA256

| TLS_AES_128_GCM_SHA256

| compressors:

| cipher preference: server

| TLSv1.2:

| ciphers:

| ECDHE-RSA-AES256-GCM-SHA384 (TLS1.2)

| ECDHE-RSA-AES128-GCM-SHA256 (TLS1.2)

| compressors:

|_ least strength: A

---

7. Findings

- **Encryption:** TLS handshake succeeds and the server negotiates modern protocols (TLS 1.3 preferred, TLS 1.2 fallback available).
- **Ciphers:** Strong AEAD ciphers available (AES-GCM / CHACHA20_POLY1305).
- **Certificate:** Self-signed certificate is valid for the configured period (Aug 12, 2025 → Aug 11, 2026). Browsers will show warnings because this certificate is not signed by a trusted CA (expected for lab).
- **Browser behavior:** Browsers will require an explicit trust exception to proceed. This is acceptable for lab testing but not for production.
- **Hardening:** Legacy protocols (SSLv2/3, TLS1.0/1.1) have been disabled. HSTS and other security headers are ready in configuration (enable carefully in production).
- **Automation:** Certbot/Let's Encrypt could not be used due to no public domain — if a public domain becomes available, shift to a CA-signed cert and enable auto renewal.

---

8. Recommendations (next steps / production considerations)

1. **Use a trusted CA certificate for production.** If you obtain a domain, use Let's Encrypt (free) or an organizational CA; automate renewal with certbot renew or equivalent.
2. **Enable OCSP stapling** in production to improve revocation check performance and privacy (requires a real CA-signed certificate and proper resolver config).
3. **HTTP/2 and HTTP/3:** Consider enabling HTTP/2 (already safe) for performance. HTTP/3 requires additional components; evaluate only if needed.
4. **HSTS deployment:** Set HSTS header in production only after ensuring that HTTPS works flawlessly for all subdomains and services; consider submitting to HSTS preload only after full validation.

5. **Certificate management:** Set monitoring/alerts for certificate expiry (external monitoring or local scripts).
6. **Periodic scans:** Run periodic SSL/TLS scans (e.g., Qualys SSL Labs, sslyze) to detect regressions or new vulnerabilities.
7. **Key protection:** Keep private key file permissions restricted (chmod 600 / owned by root) and avoid storing backups in insecure locations.
8. **Transition to CA-signed cert:** When domain available, replace the self-signed cert with Let's Encrypt cert and verify automated renewal works (certbot renew --dry-run).

---

9. Risk / limitations

- **Self-signed certificate** is inherently untrusted by client browsers — suitable only for lab/testing.
- **No OCSP stapling or real CA features** in place because there is no CA-signed certificate.
- **Automated external validation** (e.g., SSL Labs grade) cannot be performed without a public domain/IP reachable from the internet.

---

10. Appendix — Commands & artifacts used

Install & enable SSL module

sudo apt update

sudo apt install -y apache2 openssl ssl-cert

sudo a2enmod ssl headers

sudo systemctl restart apache2

Create self-signed certificate

sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \

-keyout /etc/ssl/private/apache-selfsigned.key \

-out /etc/ssl/certs/apache-selfsigned.crt \

-subj "/C=PK/ST=Punjab/L=Lahore/O=CyberSecLab/OU=IT/CN=localhost/emailAddress=test3442234@gmail.com "

sudo chmod 600 /etc/ssl/private/apache-selfsigned.key

Create and enable SSL vhost

```
# create /etc/apache2/sites-available/ssl-test.conf (see snippet above)

sudo a2ensite ssl-test.conf

sudo systemctl reload apache2
```

Verify

```
apache2ctl -M | grep ssl

openssl x509 -in /etc/ssl/certs/apache-selfsigned.crt -noout -text | head -n 40

openssl s_client -connect localhost:443 -servername localhost -showcerts

curl -vk https://localhost

nmap --script ssl-enum-ciphers -p 443 localhost
```

---

11. Conclusion

The lab deployment demonstrates a correct and secure SSL/TLS configuration for the Apache web server using a self-signed certificate. All major objectives — certificate creation, TLS hardening, and verification — were met. For submission, this configuration and the evidence above validate that encrypted connections are functioning and that modern protocols and ciphers are in use. To be production-ready, transition to a CA-signed certificate and implement monitoring/automation for certificate issuance and renewal.
